

The next line makes the label visible. Widgets are always created hidden so that we can customize them before showing them, thereby avoiding flicker.

The last line passes control of the application on to Qt. At this point, the program enters the event loop. This is a kind of stand-by mode where the program waits for user actions such as mouse clicks and key presses. User actions generate events (also called “messages”) to which the program can respond, usually by executing one or more functions. For example, when the user clicks a widget, a “mouse press” and a “mouse release” event are generated. In this respect, GUI applications differ drastically from conventional batch programs, which typically process input, produce results, and terminate without human intervention.

Output in Windows 7

Playing around with the code

To get a better understanding of how this works, let's try to tweak some of the lines

- Replace the line
- `QLabel *label = new QLabel("Hello Digit!");`
-
- with
- `QLabel *label = new QLabel("<h3><b>Hello</b> "`
- `"<font color=blue><i>Digit!</i></font></h3>");`
-
- and rebuild the application.
- The tweaked output

Connections

Now that we know how to get an output, we will now look at how to respond to user actions. The application consists of a button that the user can click to quit. The source code is very similar to “Hello Digit!”, except that we are using a `QPushButton` instead of a `QLabel` as our main widget, and we are connecting a user action (clicking a button) to a piece of code.

- `#include <QApplication>`
- `#include <QPushButton>`
- `int main(int argc, char *argv[])`